https://github.com/mdcruz/pact-js-example or check out appendix C for the answers.

10.4 Webhooks

The final concept that we want to cover in this chapter is the use of webhooks in Pact. Until now, we've been working on a single GitHub project containing both the consumer and provider workflows, so every change in our contract triggers the provider verification because it was running in a single workflow. However, consumer code isn't co-located with the provider code in real-world projects.

If the consumer and provider are in separate GitHub projects and separate workflows, when the consumer makes changes to the contract, the relevant provider verification job won't be triggered automatically, which can cause the consumer workflow to fail on the can-i-deploy job. This happens because Pact can't pre-verify the contract because the contract has changed. To work around this problem, you can ask a team member from the provider team to manually trigger the provider verification job, and when the job passes, you can rerun the can-i-deploy job on the consumer pipeline. However, this workaround is clearly not ideal!

Instead, we'd like the provider verification job to be triggered automatically when the consumer has changed something in the contract. We don't need to rerun the entire provider pipeline; we want to run just the provider verification job and publish the results to our Pact Broker. Pact webhooks can help us achieve this setup as visualized in figure 10.15. According to the Pact documentation (https://docs.pact.io/pact_broker/webhooks):

> Webhooks allow you to trigger an HTTP request when a pact is changed, a pact is published, or a verification is published. The most common use case for webhooks is to trigger a provider build every time a pact changes, and to trigger a consumer build every time a verification is published.